QueryLensAI — Interview Prep Guide

This is a companion to `README.md`. It turns the project into interview material: a memorizable pitch, then Easy → Medium → Hard questions with grounded answers, a live "spot the bug" code-review round based on real issues found in the codebase, and a behavioral round. Everything here is tied back to actual files so your answers can cite specifics instead of speaking in generalities.

Table of Contents

1. The 60-Second Pitch
2. How to Use This Guide
3. Easy Questions
4. Medium Questions
5. Hard / System-Design Questions
6. Spot-the-Bug Code Review Round
7. Behavioral / Project-Ownership Questions
8. Quick-Reference Cheat Sheet

The 60-Second Pitch

Use this as your answer to *"Tell me about a project you've worked on."*

> "QueryLensAI is a conversational data-querying platform — the idea is that instead of writing SQL or pandas code, you pick a dataset, ask a question in plain English, like 'what's the average income for states with a population over a million', and get back a natural-language answer that's actually been computed against the real data. The frontend is a React 18 and Vite single-page app with Clerk for authentication and Tailwind/DaisyUI for styling. It talks to a FastAPI backend over REST — that backend uses LangChain to orchestrate an agent that decides between retrieving context with FAISS and running actual pandas computations on the dataset, then a fine-tuned LLM composes the final answer. What I think is the most interesting part is that every generated answer is checked against a manually curated ground-truth reference before it's shown to

> the user, so the UI can display a 'Valid' or 'Invalid' badge with a plain-English reason — it's a self-auditing system, not just a chatbot. I worked on [your specific layer — frontend/backend/mobile], and one of the trickiest problems was [pick one real challenge, e.g. getting the agent to reliably choose between retrieval and computation, or getting the chat UI's state management right without a global store]."

> "QueryLensAI allows users to query structured datasets using natural language instead of writing SQL or pandas code.The React frontend sends the query to our FastAPI backend. On the backend, LangChain acts as the orchestration layer: the agent determines whether the question requires contextual retrieval through FAISS, actual dataset computation through pandas, or both.FAISS performs semantic retrieval using vector embeddings, while pandas performs deterministic operations such as filtering, aggregation and averages on the actual dataset. The LLM then converts those results into a natural-language response.Finally, we validate the generated answer against curated ground truth and return the answer along with a Valid/Invalid status and reason to the frontend."

Swap in your actual role/contribution using the Behavioral section below.

How to Use This Guide

- **Easy** = fundamentals any contributor should be able to answer about their own project (React basics, what a library does, why a tool was chosen).
- **Medium** = requires connecting two or more parts of the system together (frontend ↔ backend contract, why an architectural choice was made).
- **Hard** = system design / trade-off discussions with no single correct answer — these are where you demonstrate seniority.
- **Spot-the-Bug** = real issues discovered while reviewing this exact codebase, formatted as live code-review prompts.

Easy Questions

Q1. What does QueryLensAI do, in one sentence? A conversational AI platform that lets a user ask natural-language questions about a dataset and get back a validated, explainable answer instead of writing SQL/pandas by hand.

Q2. What does this specific repository contain? Only the frontend — a React 18 single-page app built with Vite. The AI/data backend (FastAPI, LangChain, FAISS, pandas) is a separate service this app calls over REST (see `config.js`).

Q3. Why Vite instead of Create React App? Native ES-module dev server (near-instant start, fast HMR), Rollup-based production bundling, and a lightweight plugin model — `@vitejs/plugin-react` gives React Fast Refresh without CRA's heavier webpack setup.

Q4. How is routing implemented? `react-router-dom` v7. `main.jsx` defines a `RoutesObj` array of `{ path, element }` pairs and maps it into `<Route>` elements inside one `<Routes>` — a small data-driven routing pattern instead of manually writing every route tag.

Q5. What is Clerk, and where is it wired in? A hosted authentication-as-a-service provider. `ClerkProvider` wraps the whole app in `main.jsx`; `SignedIn`/`SignedOut`/`SignInButton`/`UserButton` (in `NavBar.jsx`) render conditionally based on auth state; `useUser()` (in `DashNav.jsx`, `DashScreen.jsx`) reads the current user's name/avatar.

Q6. Is hardcoding the Clerk key in `main.jsx` a security risk? No — it's a *publishable* key (`pk_...`), explicitly designed to ship to the client. The actual secret key (`sk_...`) lives only server-side/in the Clerk dashboard and is never in frontend code. It's still not great practice for environment management, but it's not a secret-leak vulnerability.

Q7. What icon library is used, and where? `lucide-react` — tree-shakeable SVG icon components. Used in `NavBar` (`AlignJustify`), `DashNav` (`ArrowRightFromLine`), `DashMenu` (`ChevronRight`, `CircleHelp`, `SlidersHorizontal`), multiple CTAs (`MoveRight`), and the whole chat UI (`Plus`, `ChevronDown`, `Check`, `Ellipsis`, `Info`, `Bot`, `Brain`, `DatabaseZap`).

Q8. What's the styling stack? Tailwind CSS (utility classes) + DaisyUI (a Tailwind plugin adding component classes like `rounded-badge`, `hero`, `carousel`, `mockup-code`, plus a theme system) + `@tailwindcss/typography` for the `.prose` long-form text on the About/Docs pages.

Q9. How does the app fetch the list of available datasets? A `useEffect` with an empty dependency array in `DashSession.jsx` runs once on mount, calls `fetch(API_ENDPOINTS.GET_DATASET)`, and stores the returned `names[]` array in state, defaulting the selection to the first entry.

Q10. What are `useState` and `useEffect` doing, conceptually? `useState` gives a component local, re-render-triggering memory (e.g. the `messages` array). `useEffect` runs side effects — like a network call — in response to a component mounting or a dependency changing, outside of the render itself.

Q11. Difference between props and state, with an example from this codebase? Props are read-only data passed down from a parent — `FeatureCard` receives `imgsrc`, `headline`, `details` from `Cards.jsx`. State is data a component owns and updates itself — `DashSession`'s `messages` array, updated via `setMessages`.

Q12. What does `afterSignOutUrl="/"` do on `ClerkProvider`? Tells Clerk where to redirect the browser after a user signs out — back to the landing page in this app.

Q13. Why two different layout components (`MainLayout`, `DashboardLayout`)? A layout-composition pattern: `MainLayout` (NavBar + footer) wraps public marketing pages; `DashboardLayout` (sticky DashNav, no footer) wraps the authenticated dashboard. Both take a `children` prop so any page can be dropped inside.

Q14. What is `lottie-react` used for? Rendering `database-animation.json` — an exported JSON animation — on the dashboard's intro screen via `<Lottie animationData={...} loop />`.

Q15. What request does sending a chat message trigger? A `POST` to `/get_answer` with JSON body `{ "query": "<user text>" }` and header `Content-Type: application/json`.

Medium Questions

Q1. Walk me through what happens end-to-end when a user asks a question. On mount, `DashSession` fetches the dataset list. The user selects a dataset, types a question, and submits — the UI immediately shows the message and a loading indicator, then `POST /get_answer` fires. On the backend (per the project's own design/docs): a custom fine-tuned model interprets intent, a LangChain agent retrieves relevant context via FAISS and/or executes pandas operations against the dataset, an LLM composes the natural-language answer, and a validation step compares that answer to a manually curated ground truth. The response — `{ answer, valid_answer, reason }` — comes back as JSON; the UI appends the bot's answer as a chat bubble and renders a Valid/Invalid badge with a "why" modal.

Q2. What is RAG (Retrieval-Augmented Generation), and where does it fit here? RAG grounds an LLM's output in retrieved, relevant external context instead of relying purely on the model's internal/parametric knowledge. Here, FAISS stores embeddings of dataset schema/metadata/documentation; the LangChain agent retrieves the top-k most similar chunks for a given query and feeds them to the LLM alongside the computed pandas result, so the answer is grounded in the actual dataset's structure rather than guessed.

Q3. Why FAISS specifically, instead of a plain SQL `LIKE` search or a managed vector DB? FAISS gives fast approximate nearest-neighbor search over dense embeddings — semantic similarity, not keyword matching — and runs in-process with no extra infrastructure to stand up, which fits a project at this scale. The trade-off against a managed vector database (Pinecone, Weaviate, pgvector) is durability, horizontal scale, and built-in multi-tenant metadata filtering, none of which FAISS gives you for free.

Q4. What is LangChain actually orchestrating here, concretely? The decision logic for "what does this question need?" — does it need schema/context retrieval (FAISS), a computed value from the dataset (a pandas dataframe agent), or a direct LLM completion — and then assembling all of that into one final prompt/response.

Q5. Why pandas specifically for the computation layer? The underlying datasets are tabular; pandas is the standard, mature API for filter/group/aggregate operations, and LangChain has an established "pandas dataframe agent" pattern that lets an LLM translate natural language into pandas operations and execute them against a loaded dataframe.

Q6. Explain the `valid_answer` / `reason` contract and why it exists. Every backend answer is checked against a manually curated "ground truth" reference for that question, producing a boolean pass/fail plus a plain-English `reason`. This turns the product into a self-auditing system — a trust signal for end users, and an accuracy-tracking/regression-testing mechanism for the team.

Q7. Why does the frontend never actually send which dataset the user selected? This is a real bug in `DashSession.jsx`: `handleSendMessage` guards the whole function on `selectedVersion` being truthy, but only serializes `{ query }` into the POST body — the selected dataset name is tracked in state and checked, but never transmitted. It's a good example of a "looks complete but isn't" bug: the state exists and gates the logic, which makes it easy to assume it's also being sent.

Q8. How would CORS need to be configured, and why? The frontend (a Vercel domain) and backend (a different host/port — `localhost:8000` in dev, a VM IP in the commented fallback) are different origins, so the FastAPI app needs `CORSMiddleware` allowlisting the frontend's origin(s), the methods used (`GET`, `POST`), and the `Content-Type` header — otherwise the browser blocks the response before JS ever sees it.

Q9. What would you change about how `config.js` handles environments? Replace the commented-out URL swap with Vite's built-in env support — `import.meta.env.VITE_API_BASE_URL` sourced from `.env.development` / `.env.production` files — so switching environments doesn't require editing tracked source code.

Q10. How does the chat UI show loading state, and what's the limitation? A `loadingMessage` boolean toggles an animated `Ellipsis` bubble while `await fetch(...)` is pending. It's a single blocking request/response with no streaming, so for a slow LLM call the user just watches the dots with no partial progress — a production version would likely stream tokens via SSE or WebSockets.

Q11. What's the intended purpose of the "final report" PDF export, and how would it technically work? `html2canvas` would walk the target DOM subtree (e.g. the chat/validation panel) and rasterize it to a canvas/PNG entirely client-side — no server-side rendering needed. `jsPDF` would then create a PDF document and place that image as a page, which the browser can save/download. It's a "screenshot-to-PDF" pattern, not structured/selectable-text PDF generation. Note: this isn't implemented in the current codebase (no dependency, no call site) — it's a designed feature, described here as intended.

Q12. Why might the product need a "custom model" step before the LLM agent, rather than sending the raw question straight to a general-purpose LLM? Per the team's own description, this is an in-house fine-tuned model — likely tuned to the vocabulary/shape of the supported datasets, deciding how to route or decompose a question before the heavier/general agent step runs. That can improve both accuracy and cost/latency versus always invoking a full agentic chain for every query.

Q13. How would you add tests to this frontend today? There currently are none configured (no test runner in `package.json`). Vitest pairs naturally with Vite; start with pure/presentational components (`FeatureCard`, `AboutCard`), then mock `fetch` to test `DashSession`'s dataset-load effect and send-message handler.

Hard / System-Design Questions

Q1. How would you scale the backend to handle many concurrent users querying large datasets with an LLM agent? Talking points: async FastAPI endpoints with a task queue (Celery/RQ, or async workers) so long LLM calls don't block request threads; cache embeddings/FAISS indices per dataset instead of rebuilding per request; stream responses (SSE/WebSocket) instead of one blocking round trip; rate-limit per user/session; scale stateless API instances horizontally behind a load balancer while keeping the vector store and dataframes in shared/persisted storage rather than per-process memory.

Q2. LangChain's pandas dataframe agent can generate and execute code against a live Python process. What's the security concern, and how do you mitigate it? This is a code-execution / prompt-injection risk — an adversarial query could try to make the agent execute something outside the intended dataframe operations. Mitigations: run the agent in a sandboxed, least-privilege process; allow-list permitted pandas operations instead of arbitrary `exec`; enforce timeouts and resource limits; block filesystem/network access from within the sandbox; validate and constrain user input before it reaches the prompt.

Q3. How would you evolve "manual ground truth" validation so it doesn't require a human to hand-write a reference answer for every possible question? Options to discuss: LLM-as-judge (a second model scores the primary answer against a rubric instead of requiring an exact reference); embedding-based semantic-similarity thresholds between generated and reference answers; tolerant numeric comparison for quantitative answers; a growing "golden dataset" of Q&A pairs for regression testing whenever the model/prompt changes; a human-review queue for low-confidence verdicts that feeds back into that golden set.

Q4. How would you design multi-tenant data isolation if different organizations used this platform with their own private datasets? Per-tenant namespacing (or fully separate indices) for FAISS and dataframe storage; never share embeddings/context across tenants during retrieval; scope every backend request by an authenticated tenant/session identity — Clerk already gives a verifiable user identity on the frontend, so you'd propagate that as a verified token to the backend and check it there (not currently happening based on this repo's code); encrypt sensitive data at rest.

Q5. How would you introduce API versioning safely, given `config.js` already anticipates `/v1/get_answer` and `/v2/get_answer`? Keep `v0`'s contract frozen for existing clients; add `v1`/`v2` as additive, parallel routes rather than mutating `v0`; document a deprecation window; let the frontend opt into a newer version deliberately rather than a hard cutover; add contract tests so a backend change can't silently break the `{ answer, valid_answer, reason }` shape the frontend depends on.

Q6. The current request is a single blocking `await fetch()`. How would you redesign this for a streaming LLM experience? Replace the single JSON response with Server-Sent Events or a WebSocket channel; stream partial tokens as they're generated and append them to the last "bot" message incrementally; keep the ground-truth validation step as a distinct final event once the full answer is assembled (validation needs the complete answer, not partial tokens); handle reconnects/backpressure on the client.

Q7. What are the trade-offs of FAISS versus a managed vector database as this product grows? FAISS: fast, free, in-process, zero extra ops — but no built-in persistence, replication, horizontal scale, or native multi-tenant metadata filtering. Managed options (Pinecone, Weaviate, pgvector): built-in persistence/scale/filtering/access control, at the cost of money and an external network dependency. FAISS is pragmatic for a handful of curated datasets; a managed store becomes more attractive with many tenants or huge corpora.

Q8. How would you monitor and debug a wrong/invalid answer in production? Log the full pipeline trace per request (retrieved chunks, generated pandas operations, raw LLM output, validation result) tied to a request ID; surface `valid_answer: false` cases into a review dashboard; track the invalid-rate over time as a core product health metric; replay historically-failing queries against newer model/prompt versions before shipping changes.

Spot-the-Bug Code Review Round

Real findings from this exact codebase, framed as "here's a snippet — what's wrong, and how would you fix it?"

1. Dataset selection never reaches the backend

```
// DashSession.jsx
const handleSendMessage = async () => {
    if (inputText.trim() !== "" && selectedVersion) {
        // ...
        const response = await fetch(API_ENDPOINTS.GET_ANS_V0, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ query: inputText }), // selectedVersion is
missing here
        });
```

What's wrong: `selectedVersion` gates the function but is never included in the request body. *Fix:* `body: JSON.stringify({ query: inputText, dataset: selectedVersion })` (and update the backend contract to accept it).

2. The chat workspace can't actually be reached

```
// DashScreen.jsx
const [open, setOpen] = useState(false);
// ...
<a href="https://http://localhost:3000/">
  <button>Get Started</button>
</a>
// setOpen is never called anywhere in this file
```

What's wrong: Two issues stacked — a malformed URL (`https://http://...` has two protocols) on the CTA, and `setOpen(true)` is never called, so `{!open ? <intro/> : <DashSession/>}` always renders the intro. *Fix:* Replace the anchor with a button `onClick={() => setOpen(true)}`.

3. Inconsistent router import

```
// Landing.jsx
import { Link } from 'react-router';       // <- everywhere else it's 'react-
router-dom'
```

What's wrong: Works today only because `react-router-dom@7` depends on `react-router` internally — fragile, and inconsistent with the rest of the codebase. *Fix:* Standardize on `react-router-dom` everywhere.

4. Orphaned page

```
// main.jsx — RoutesObj array
{ path: '/', element: <App /> },
{ path: '/login', element: <Login /> },
{ path: '/Docs', element: <Docs /> },
{ path: '/About', element: <About /> },
{ path: '/Dashboard', element: <Dashboard /> },
// no route for Registration.jsx
```

What's wrong: `Registration.jsx` exists in `src/Pages/Auth/` but is never imported or routed. *Fix:* Either add a `/register` route or delete the dead file.

Behavioral / Project-Ownership Questions

Q: What was your specific role/contribution? Tailor this to whichever real layer you owned — this project's own team split was: UI/UX + full-stack (frontend architecture and product design), backend/API + LLM integration (server-side design and metadata-driven query processing), and cross-platform/web (extending reach across devices). Speak concretely about the layer you touched — specific files, specific decisions — rather than generic project-management language.

Q: What was the hardest technical problem you ran into? Strong, grounded answers to adapt: getting an agent to reliably choose between "retrieve context" and "compute a value" for ambiguous natural-language questions; keeping FAISS-retrieved context relevant enough that the LLM doesn't answer from stale/irrelevant chunks; designing ground-truth validation so it doesn't degrade into brittle exact-string matching; managing chat UI state cleanly with only local `useState` as the number of flags grew (this file alone tracks eleven pieces of state).

Q: What would you do differently if you rebuilt this today? Use the Known Gaps & Tech Debt list as authentic material: send the selected dataset with every query, wire up the dashboard's entry point, move secrets/URLs into `.env` files, add response streaming, add a test suite, consolidate the duplicated scrollbar CSS.

Q: How did a small team divide the work without stepping on each other? Discuss splitting by layer (frontend/UI vs. backend/API vs. cross-platform) with a shared contract at the boundary — in this project, that boundary is the `{ query } → {`

`answer, valid_answer, reason }` REST contract, which let frontend and backend work independently as long as both sides honored that shape.

Q: How do you decide what's in scope vs. a "nice to have" under a deadline? Use the PDF export feature as a live example: it's part of the designed pipeline and shows up in the product's own description of itself, but it's explicitly not implemented yet — a good example of consciously sequencing "core loop first" (ask → validate → answer) before "delight" features (export/share).

Quick-Reference Cheat Sheet

Term	One-line definition
Clerk	Hosted authentication-as-a-service; handles sign-in/out, sessions, and user profile UI components
FAISS	Facebook AI Similarity Search — a library for fast approximate nearest-neighbor search over vector embeddings
LangChain	A framework for orchestrating LLM calls, tools, and agents into multi-step pipelines
RAG	Retrieval-Augmented Generation — grounding an LLM's answer in retrieved external context instead of relying on its own memory
Agent (LLM context)	An LLM-driven controller that decides which tool(s) to call (retrieval, computation, etc.) to answer a request
Vector embedding	A numeric representation of text/data such that semantically similar items are close together in vector space
Ground truth	A known-correct reference answer used to check a generated answer's accuracy
pandas dataframe agent	A LangChain pattern where an LLM translates natural language into pandas operations executed against a real dataframe
Vite	A fast frontend build tool using native ES modules in dev and Rollup for production bundling
HMR	Hot Module Replacement — updates modules in the browser without a full page reload

Term	One-line definition
DaisyUI	A Tailwind CSS plugin adding pre-built component classes and theming on top of utility classes
CORS	Cross-Origin Resource Sharing — browser security policy that a backend must explicitly allow for cross-origin requests to succeed
SSE	Server-Sent Events — a one-way streaming protocol from server to client over HTTP, well suited to streaming LLM tokens
ESLint flat config	The newer, array-based configuration format for ESLint (replacing `.eslintrc`), used in `eslint.config.js` here
html2canvas	Client-side library that rasterizes a DOM subtree into a canvas/image
jsPDF	Client-side library for generating PDF documents in the browser, often combined with`html2canvas` for "export as PDF"

Explain It Like I'm 8 🍕

Use this version when someone (or a very tired interviewer) wants the *simplest possible* version of the story — no jargon, just a picture in your head. It's the same project, just told like a bedtime story. Once you've got this, all the harder Q&A above is just "the grown-up words for the same story."

The Pizza Shop Story

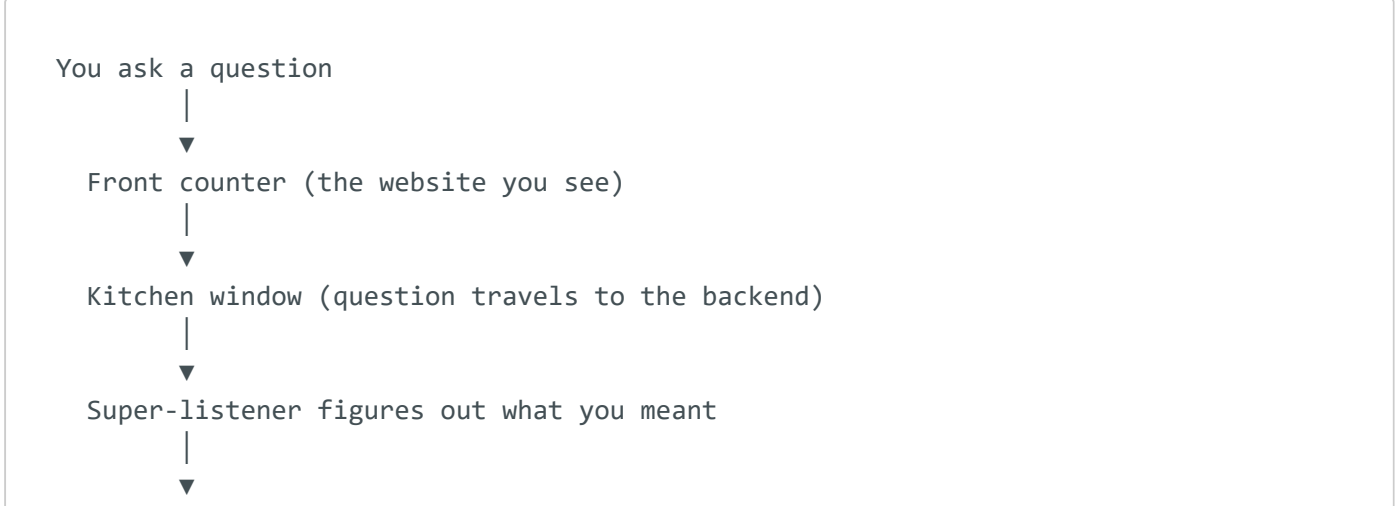
Imagine QueryLensAI is a **pizza shop**. 🍕

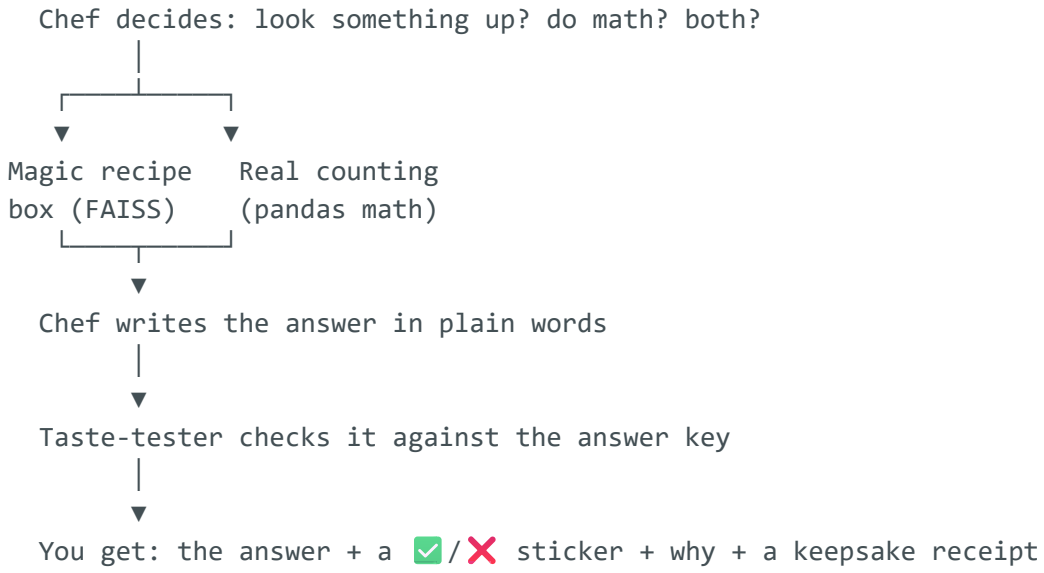
- **You** are a hungry customer with a question, like *"How many pepperoni pizzas did we sell last month?"*
- You don't ask in a weird secret code (like SQL) — you just ask like a normal person.
- There's a **front counter** (that's the part of the shop you can see and touch — the **frontend**, the website).
- Behind a door there's a **kitchen** you never see (that's the **backend** — all the real work happens back there, hidden from you).

Here's what happens to your question, step by step:

1. 🗣️ **You talk to the counter person.** You type your question into the chat box. The counter person writes your order on a little ticket and slides it through the kitchen window. *(The frontend sends your question to the backend.)*
2. 👂 **A super-listener reads your ticket.** This is a helper who's really good at understanding orders even if you say them a bit weirdly — they figure out exactly what you *really* meant. *(The "custom model" interprets your question.)*
3. 🧑‍🍳 **The head chef decides what to do.** Does the chef need to peek in the fridge for an ingredient, or actually cook/count something? The chef is like a smart detective who picks the right tools for the job. *(This is the "LLM agent" deciding what to do.)*
4. 📇 **A magic recipe box finds the right card instantly.** Instead of flipping through a thousand recipe cards one by one, there's a magic box that jumps straight to the 2-3 cards that actually matter. *(That's **FAISS** — it finds the right information fast instead of searching everything.)*
5. 🧮 **The chef actually counts the real ingredients.** No guessing — the chef opens the real fridge (the real data) and actually counts/weighs/adds things up for real. *(That's **pandas** doing the real math on the real data.)*
6. 📖 **A taste-tester checks the pizza against a secret answer key.** Before your pizza goes out, someone compares it to a "perfect pizza" photo that a teacher already made earlier. *(That's comparing the answer to the **manual ground truth**.)*
7. ✅ ❌ **You get a sticker.** If it matches the perfect photo, it gets a green sticker (**Valid**). If it doesn't quite match, it gets a red sticker (**Invalid**) *and* a little note explaining why. *(That's the **validation & comparison** step.)*
8. 🧾 **You get your pizza AND a receipt.** The receipt is something you can keep and show your friends later. *(That's the **final report** you could export/save.)*

The simplest picture





Grown-up word decoder

Kid-story word	What it really is
Front counter	The**frontend** — the website (React) you actually see and click on
Kitchen (hidden)	The**backend** — the FastAPI server doing the real work
The ticket	The API request (`POST /get_answer`)
Super-listener	The**custom fine-tuned model** that figures out what you're really asking
Head chef	The**LLM agent** (built with LangChain) deciding what steps are needed
Magic recipe box	**FAISS** — finds the most relevant info fast instead of checking everything
Real counting/measuring	**pandas** — does the actual math on the actual dataset
Perfect-pizza photo	The**manual ground truth** — an answer a person already confirmed is correct
Sticker + note	The**validation result** (`valid_answer`) and **reason** shown in the app
Receipt to keep	The**final report** (planned PDF export)

The one-breath summary

"You ask a question like you're talking to a person. A website sends it to a hidden kitchen. Someone figures out what you meant, a smart helper decides whether to look something up or do real math, then checks the answer against one a teacher already approved — and you get your answer back with a sticker saying if it's correct, plus a receipt to keep."

QueryLensAI — Complete Architecture & Interview Breakdown

> **Conversational Data Querying + RAG + Agentic AI + Deterministic Computation + Answer Validation**

---

1. What is QueryLensAI?

The easiest way to understand **QueryLensAI** is to stop thinking about **LangChain, Agents, FAISS, Pandas, RAG, embeddings, and LLMs** as separate buzzwords.

They form **one pipeline**.

QueryLensAI is a conversational data-querying platform where a user can select a dataset and ask questions about it using **natural language**, instead of manually writing SQL queries or Pandas code.

For example, imagine we have this dataset:

```
State          Population      Income
---------------------------------------
West Bengal    91,000,000      12000
Karnataka      61,000,000      18000
Goa             1,500,000      25000
```

Normally, to find the average income of states with a population greater than 50 million, someone might have to write:

```
filtered = df[df["Population"] > 50_000_000]
average = filtered["Income"].mean()
```

With QueryLensAI, the user simply asks:

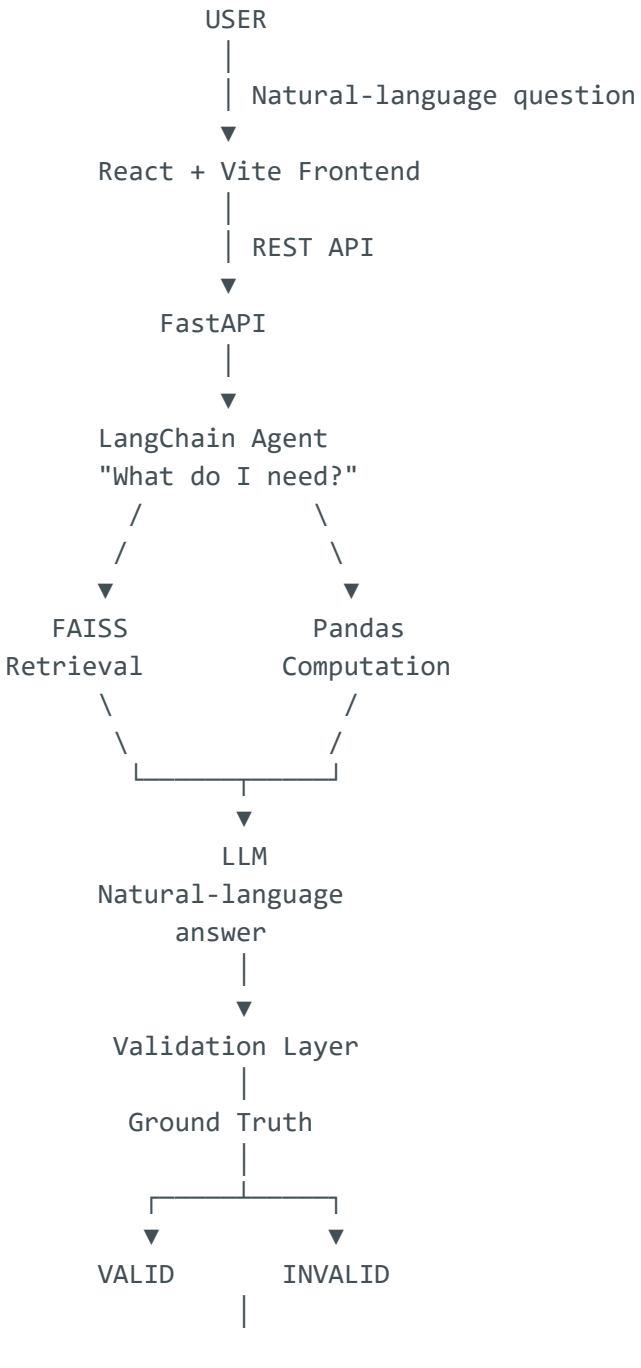
```
"What is the average income of states
with a population greater than 50 million?"
```

QueryLensAI understands the question, performs the required operation against the real dataset, generates a natural-language answer, and validates that answer before showing it to the user.

---

2. High-Level Architecture

The overall architecture can be thought of as:



```
                    ▼
                 React UI
```

The simplest mental model is:

```
React
   ↓
"What did the user ask?"

FastAPI
   ↓
"Backend entry point"

LangChain
   ↓
"Coordinate the workflow"

Agent
   ↓
"Which tool should I use?"

FAISS
   ↓
"FIND relevant information"

Pandas
   ↓
"CALCULATE using real data"

LLM
   ↓
"EXPLAIN the result"

Ground Truth
   ↓
"VERIFY the result"

React
   ↓
"SHOW answer + Valid/Invalid"
```

Remember these five lines:

```
LangChain = ORCHESTRATE
Agent     = DECIDE
FAISS     = RETRIEVE
Pandas    = CALCULATE
LLM       = EXPLAIN
```

---

3. Frontend — React + Vite

The frontend is built using:

- React 18
- Vite
- Tailwind CSS
- DaisyUI
- Clerk Authentication

Its responsibility is primarily:

```
User Login
    ↓
Select Dataset
    ↓
Enter Natural Language Question
    ↓
Send REST Request
    ↓
Show Loading State
    ↓
Receive Backend Response
    ↓
Display Answer
    ↓
Display Valid / Invalid Status
```

The frontend should **not perform the actual dataset computation**.

Instead, it sends the user's request to the backend.

Conceptually:

```
const response = await fetch("/api/query", {
    method: "POST",
    headers: {
        "Content-Type": "application/json"
    },
    body: JSON.stringify({
        dataset: selectedDataset,
        query: userQuestion
    })
});
```

---

4. Why Vite?

Vite is the frontend build and development tool.

It provides:

- Fast development server
- Fast hot-module replacement
- Optimized production builds
- Simple environment-variable handling
- Good React integration

So:

```
React = UI library

Vite = Development/build tooling
```

They solve different problems.

---

5. Authentication — Clerk

Clerk handles user authentication.

Instead of implementing authentication completely from scratch, Clerk can handle things such as:

```
Sign Up
Login
Logout
Session Management
Authenticated User
Protected Pages
```

Conceptually:

```
User
 ↓
Clerk Login
 ↓
```

```
Authenticated Session
 ↓
React Application
 ↓
QueryLensAI Features
```

Authentication and authorization are different:

```
Authentication
     ↓
"Who are you?"

Authorization
     ↓
"What are you allowed to access?"
```

---

6. FastAPI Backend

The frontend communicates with a **FastAPI backend through REST APIs**.

FastAPI acts as the entry point for the AI/data-processing workflow.

Conceptually:

```
React
   ↓
POST /query
   ↓
FastAPI
   ↓
Validate Request
   ↓
Process Query
   ↓
LangChain / Agent
   ↓
Return Response
```

A simplified endpoint could look like:

```python
@app.post("/query")
async def query_dataset(request: QueryRequest):

    question = request.question
    dataset = request.dataset
```

```
    result = process_query(question, dataset)

    return {
        "answer": result.answer,
        "valid_answer": result.valid,
        "reason": result.reason
    }
```

FastAPI is useful here because it provides:

- REST API development
- Request validation
- Pydantic models
- Async support
- Automatic API documentation
- Clean integration with Python AI/ML libraries

---

7. Why FastAPI Instead of Flask?

A concise interview answer:

> FastAPI provides built-in request validation using Pydantic, automatic OpenAPI documentation, type hints, and strong async support. Since QueryLensAI already uses Python-based AI and data libraries, FastAPI provides a clean backend layer around those components.

---

8. The Main Problem

Now comes the interesting part.

Suppose the user asks:

```
"What is the average income?"
```

This requires **calculation**.

But suppose the user asks:

```
"What does the income column represent?"
```

This is more of a **context/retrieval question**.

And suppose the user asks:

```
"What is the average income and explain what
the income field represents?"
```

Now we potentially need:

```
Retrieval
    +
Calculation
    +
Natural-language generation
```

This is where **LangChain and the agent architecture** become useful.

---

9. What is LangChain?

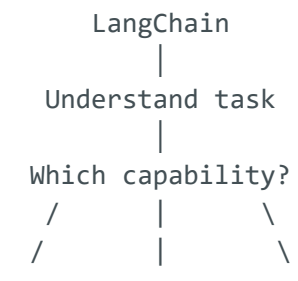
LangChain is used as the **orchestration layer** around the LLM and available tools.

The key word is:

ORCHESTRATION

Instead of asking an LLM to directly answer every question, the system can coordinate different capabilities.

For example:



```
          ▼          ▼          ▼
       FAISS      Pandas      LLM
       Search     Calculate   Generate
```

---

10. What Does "Orchestration" Mean?

Think about an orchestra.

There are:

```
Piano
Violin
Drums
Flute
```

The conductor doesn't personally play every instrument.

Instead, the conductor coordinates them:

```
"Piano plays now."

"Violin joins."

"Drums start."

"Everyone together."
```

That is orchestration.

Similarly:

```
FAISS
Pandas
LLM
Other tools
```

are different capabilities.

LangChain helps coordinate how these components participate in answering the user's question.

---

11. Why LangChain?

Without an orchestration layer, we could manually write logic such as:

```
if question_needs_calculation:
    use_pandas()

elif question_needs_information:
    use_faiss()

else:
    use_llm()
```

This works for simple cases.

But real user questions can contain multiple intents.

For example:

```
"Find the state with the highest income,
compare it with the average income,
and explain what the income field represents."
```

Now the system may need to:

```
1. Understand the question
2. Identify the relevant columns
3. Retrieve contextual information
4. Run calculations
5. Combine the results
6. Generate a natural-language answer
```

LangChain provides abstractions for coordinating these steps.

---

12. What is an AI Agent?

An **agent** is essentially an LLM-driven decision-making component that can decide which available tool or action should be used.

Suppose we give the agent:

```
Tool 1 → FAISS Search

Tool 2 → Pandas Computation
```

Then the user asks:

```
"What does annual_income mean?"
```

The agent may decide:

```
This is a contextual question.

→ Use FAISS
```

But if the user asks:

```
"What is the average annual income?"
```

The agent can decide:

```
This requires actual computation.

→ Use Pandas
```

If the user asks:

```
"Calculate the average annual income
and explain what this field represents."
```

Then:

```
FAISS
   +
Pandas
   +
LLM
```

may all participate.

---

13. LangChain vs Agent

This distinction is important.

```
LangChain
     ↓
Framework / orchestration infrastructure

Agent
     ↓
Decision-making component
```

A simple interview explanation:

> LangChain provides the framework for connecting the LLM with external tools and workflows, while the agent uses the LLM's reasoning to decide which available tool should be called for a particular query.

---

14. Why Not Just Send Everything to the LLM?

Because an LLM should **not be treated as the source of truth for structured numerical computation**.

Suppose the dataset contains:

```
Income
------
100
200
300
400
```

We don't want the LLM to guess the average.

Instead:

```
df["Income"].mean()
```

gives:

```
250
```

This is deterministic.

So the architecture separates responsibilities:

```
LLM
 ↓
Understand + Explain

Pandas
 ↓
Compute

FAISS
 ↓
Retrieve

Validation Layer
 ↓
Verify
```

A strong interview answer:

> We don't rely on the LLM for deterministic numerical operations because LLMs can hallucinate or make calculation errors. Pandas performs the actual operation against the real dataset, while the LLM focuses on understanding the user's intent and presenting the result naturally.

---

15. What is Pandas Doing?

Pandas performs actual operations against structured data.

For example:

```
State          Population      Income
-------------------------------------
```

```
West Bengal    91M             12000
Karnataka      61M             18000
Goa             1.5M           25000
```

Question:

```
"What is the average income for states
with population above 50 million?"
```

Pandas can perform:

```
filtered = df[df["Population"] > 50_000_000]

average_income = filtered["Income"].mean()
```

Result:

```
(12000 + 18000) / 2

= 15000
```

So:

```
Pandas = deterministic structured-data computation
```

Examples include:

```
Filtering
Sorting
Grouping
Average
Maximum
Minimum
Sum
Count
Aggregation
Column selection
Statistical operations
```

---

16. What is FAISS?

FAISS stands for:

Facebook AI Similarity Search

FAISS is a library designed for efficient **vector similarity search**.

In simple terms:

> FAISS helps us find information that is semantically similar to the user's question.

The key word is:

SEMANTIC

Semantic means:

```
Meaning
```

rather than simply:

```
Exact matching
```

---

17. Keyword Search vs Semantic Search

Suppose our metadata contains:

```
annual_income = yearly salary of the customer
```

The user asks:

```
"What are the customer's earnings?"
```

A simple keyword search might compare:

```
earnings
```

with:

```
annual_income
```

and potentially fail to recognize the relationship.

But semantically:

```
earnings
≈
salary
≈
income
```

These concepts are related.

Vector embeddings allow us to capture this similarity.

---

18. What is an Embedding?

An embedding is a **numerical vector representation of information**.

For example:

```
"salary"
```

might conceptually become:

```
[0.81, 0.73, 0.16, 0.42, ...]
```

And:

```
"income"
```

might become:

```
[0.80, 0.71, 0.17, 0.40, ...]
```

Whereas:

```
"elephant"
```

might become something very different:

```
[0.11, 0.08, 0.92, 0.13, ...]
```

The exact numbers aren't important.

The idea is:

```
Similar meaning
      ↓
Similar vectors
      ↓
Closer in vector space
```

Therefore:

```
salary ↔ income
```

should be relatively close.

While:

```
salary ←----------------→ elephant
```

should be farther apart.

19. How FAISS Works

Suppose we have:

```
Dataset metadata
      ↓
Break into chunks
      ↓
Embedding Model
      ↓
Vectors
      ↓
Store in FAISS
```

Example:

```
"Population represents number of residents"
                      ↓
               Embedding Model
                      ↓
          [0.21, 0.42, 0.77, ...]

"Income represents annual salary"
                      ↓
               Embedding Model
                      ↓
          [0.82, 0.11, 0.34, ...]
```

FAISS indexes those vectors.

Now the user asks:

```
"What field tells me how much someone earns?"
```

The question is also embedded:

```
Question
   ↓
Embedding Model
   ↓
Query Vector
```

FAISS performs a nearest-neighbor search:

```
Query Vector
     ↓
Compare against stored vectors
     ↓
Find nearest vectors
     ↓
Return relevant context
```

It might retrieve:

```
"Income represents annual salary"
```

even though the user never used the exact word **income**.

---

20. Why FAISS Instead of SQL?

Because they solve different problems.

SQL

Excellent for:

```
WHERE
JOIN
GROUP BY
ORDER BY
COUNT
SUM
AVG
Structured filtering
```

Example:

```sql
SELECT AVG(income)
FROM states
WHERE population > 50000000;
```

FAISS

Excellent for:

```
Semantic search
Meaning-based retrieval
Finding related information
Vector similarity
```

So:

```
SQL/Pandas
     ↓
"What is the average income?"

FAISS
     ↓
"What does income mean?"
```

They are complementary technologies.

---

21. Why FAISS Instead of a Normal Database?

A normal relational database is designed primarily around structured data and exact conditions.

FAISS is specifically optimized for:

```
Nearest-neighbor vector search
```

So if we have thousands or millions of embeddings, FAISS can efficiently answer:

```
"Which stored vectors are closest to this query vector?"
```

---

22. What is RAG?

RAG stands for:

Retrieval-Augmented Generation

The idea is very simple.

Instead of doing:

```
Question
   ↓
LLM
   ↓
Answer from model knowledge
```

we do:

```
Question
   ↓
Retrieve relevant information
   ↓
Give Question + Context to LLM
   ↓
Generate grounded answer
```

That's RAG.

---

23. Why Do We Need RAG?

An LLM has knowledge from its training data.

But it doesn't automatically know:

```
Our uploaded dataset
Our dataset schema
Our private information
Our latest information
Our business-specific definitions
```

So we retrieve relevant external information and provide it to the model.

Conceptually:

```
User:
"What does customer income mean?"
              ↓
         Create embedding
              ↓
           FAISS
              ↓
Retrieve:
"customer_income represents annual income"
              ↓
           Prompt
              ↓
Question + Retrieved Context
              ↓
             LLM
              ↓
Natural-language answer
```

---

24. FAISS vs RAG

Do not confuse these.

```
FAISS
   ↓
Technology used for vector retrieval

RAG
   ↓
Architecture/pattern where retrieval
is used before generation
```

So:

> FAISS can implement the retrieval component of a RAG system.

---

25. What is the LLM Doing?

The LLM's primary responsibilities are around:

```
Understanding natural language
Understanding intent
Reasoning about requested actions
Generating natural-language responses
```

Suppose Pandas returns:

```
15000
```

We don't necessarily want to show:

```
15000
```

The LLM can produce:

```
"The average income of states with populations
above 50 million is 15,000."
```

So:

```
Pandas
  ↓
Produces factual/computational result

LLM
  ↓
Produces understandable explanation
```

---

26. What About the Fine-Tuned Model?

The project architecture also describes a custom/fine-tuned model being involved in interpreting or routing user questions.

The safe way to explain this in an interview is:

> The custom model helps interpret domain-specific user intent before the broader agent workflow. This can help make routing more reliable for the kinds of dataset

> questions QueryLensAI expects.

Conceptually:

```
User Question
      ↓
Custom/Fine-Tuned Model
      ↓
Understand Intent
      ↓
Agent Workflow
      ↓
Choose Appropriate Tool
```

Do not claim specific fine-tuning training data, algorithms, or performance characteristics unless you actually implemented and measured them.

---

27. Complete Query Example

Let's walk through one complete example.

Dataset:

```
State          Population      Income
---------------------------------------
West Bengal    91M             12000
Karnataka      61M             18000
Goa             1.5M           25000
```

User asks:

```
"What is the average income of states
with population greater than 50 million?"
```

Step 1 — Frontend

React captures:

```
Selected Dataset
+
User Question
```

and sends it to FastAPI.

Step 2 — FastAPI

FastAPI:

```
Receives request
    ↓
Validates request
    ↓
Passes query into AI/data workflow
```

Step 3 — Intent Understanding

The system identifies:

```
Operation = average

Column = Income

Condition:
Population > 50 million
```

Step 4 — Agent Decision

The agent recognizes:

```
This requires deterministic computation.
```

Therefore:

```
Use Pandas
```

Step 5 — Pandas

Pandas filters:

```
West Bengal → 91M → 12000
Karnataka   → 61M → 18000
```

Goa is excluded.

Then:

```
Average

= (12000 + 18000) / 2

= 15000
```

Step 6 — LLM

The result is passed into the generation layer.

The LLM produces:

```
"The average income of states with populations
greater than 50 million is 15,000."
```

---

28. Ground-Truth Validation

This is one of the most important parts of QueryLensAI.

The system doesn't simply trust every generated response.

Instead, the generated answer is compared against a **manually curated ground-truth reference**.

Conceptually:

```
Generated Answer
       ↓
Validation Layer
       ⇔
Ground Truth
       ↓
Does it match?
     /       \
   YES        NO
    ↓          ↓
 VALID      INVALID
```

The backend can return something like:

```
{
    "answer": "The average income is 15,000.",
    "valid_answer": true,
    "reason": "The generated answer matches the expected result."
}
```

---

29. Why Ground Truth?

LLMs can hallucinate.

For example, actual Pandas result:

```
15000
```

But suppose the generated response accidentally says:

```
"The average income is 18,000."
```

Without validation:

```
Wrong answer
     ↓
Shown to user
```

With validation:

```
Generated answer
      ↓
Ground-truth comparison
      ↓
Mismatch detected
      ↓
INVALID
```

Therefore, validation adds a **trust layer** around the AI output.

---

30. Why Is QueryLensAI More Than a Chatbot?

A normal chatbot might look like:

```
Question
   ↓
LLM
   ↓
Answer
```

QueryLensAI looks more like:

```
Question
   ↓
Intent Understanding
   ↓
Agent Decision
   ↓
Retrieval / Computation
   ↓
Real Dataset
   ↓
LLM Generation
```

```
   ↓
Ground-Truth Validation
   ↓
Answer + Trust Signal
```

Therefore, the LLM is **one component of the system**, rather than the entire system.

---

31. What Does "Self-Auditing" Mean?

The project can be described as self-auditing because it doesn't simply generate an answer.

It also evaluates the generated answer against a known reference and exposes the result.

```
Generate
   ↓
Verify
   ↓
Explain validation result
```

The UI can therefore show:

```
Answer:
The average income is 15,000.

✓ VALID

Reason:
The answer matches the expected result.
```

or:

```
Answer:
The average income is 18,000.

✗ INVALID

Reason:
The calculated result does not match
the expected reference value.
```

---

32. End-to-End Request Flow

The complete flow can be summarized as:

```
1. User logs in through Clerk

              ↓

2. User selects a dataset

              ↓

3. User asks a natural-language question

              ↓

4. React sends REST request to FastAPI

              ↓

5. FastAPI validates request

              ↓

6. Query enters LangChain workflow

              ↓

7. Agent understands the required action

              ↓

8. Agent chooses:

       FAISS
         or
       Pandas
         or
       Combination

              ↓

9. Relevant context/results are obtained

              ↓

10. LLM generates natural-language response

              ↓

11. Response is checked against ground truth

              ↓

12. Backend returns:
```

```
        Answer
        Valid / Invalid
        Reason

               ↓

13. React displays everything to the user
```

---

33. Example — Retrieval Question

User asks:

```
"What does annual income represent?"
```

Flow:

```
Question
   ↓
Agent
   ↓
Recognizes retrieval requirement
   ↓
Create query embedding
   ↓
FAISS similarity search
   ↓
Retrieve relevant metadata
   ↓
LLM
   ↓
Generate explanation
   ↓
Validation
   ↓
Frontend
```

---

34. Example — Computation Question

User:

```
"What is the average annual income?"
```

Flow:

```
Question
   ↓
Agent
   ↓
Recognizes computation
   ↓
Pandas
   ↓
df["income"].mean()
   ↓
Actual numerical result
   ↓
LLM explains
   ↓
Validation
   ↓
Frontend
```

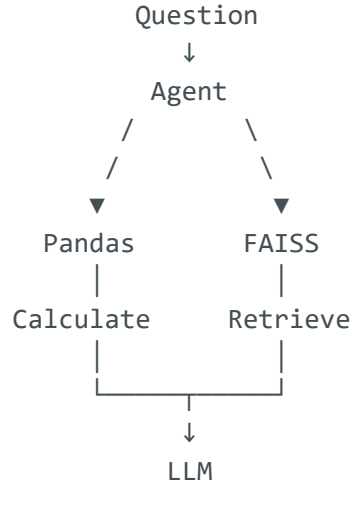
---

35. Example — Hybrid Question

User:

```
"What is the average annual income,
and explain what annual income represents?"
```

Now we potentially need both:



```
                ↓
        Combined Answer
                ↓
           Validation
```

This is where an agentic architecture becomes especially useful.

---

36. Why Not Convert Natural Language Directly to SQL?

That's another possible architecture.

For example:

```
User Question
     ↓
LLM
     ↓
Generate SQL
     ↓
Database
```

But QueryLensAI is designed around dataset interaction where Pandas can directly operate over the selected structured dataset.

Pandas provides convenient operations for:

```
Filtering
Aggregation
Statistics
Data transformation
Exploration
```

A SQL-based approach could also be valid depending on how datasets are stored.

---

37. Why Not Use Only Pandas?

Because Pandas understands:

```
Rows
Columns
Filters
Aggregations
```

It doesn't naturally understand:

```
"What does this column mean?"

"Find information related to customer earnings."

"Which field is semantically related to salary?"
```

That requires natural-language understanding and potentially semantic retrieval.

Therefore:

```
Pandas = structured computation

FAISS = semantic retrieval

LLM = natural-language intelligence
```

---

38. Why Not Use Only FAISS?

Because FAISS retrieves similar vectors.

It doesn't perform arbitrary dataframe calculations such as:

```
Average income

Total revenue

Maximum population

Group by state

Count customers

Filter income > X
```

That's why Pandas is still needed.

---

39. Why Not Use Only the LLM?

Because:

```
LLM
 ↓
Probabilistic

Pandas
 ↓
Deterministic
```

For numerical operations:

```
Actual dataset
     ↓
Actual computation
```

is safer than:

```
Actual dataset
     ↓
LLM guesses/calculates
```

Therefore:

> Use deterministic tools for deterministic problems.

---

40. REST API Design

Conceptually, the frontend might send:

```
POST /api/query
```

Request:

```
{
    "dataset": "population_data",
    "question": "What is the average income?"
}
```

Response:

```
{
    "answer": "The average income is 15,000.",
    "valid_answer": true,
    "reason": "The generated result matches the reference answer."
}
```

The frontend doesn't need to know whether:

```
FAISS
Pandas
LLM
```

were internally used.

That complexity stays behind the API.

---

41. Why REST?

REST gives a clean separation between:

```
Frontend
   ↓
HTTP API
   ↓
Backend
```

Advantages include:

- Simple HTTP communication
- Clear request/response contracts
- Frontend/backend separation
- Easy testing

- Easy integration with different clients
- Stateless request handling

---

42. Why Pydantic?

FastAPI uses Pydantic models for request/response validation.

Example:

```
from pydantic import BaseModel

class QueryRequest(BaseModel):
    dataset: str
    question: str
```

If the frontend sends:

```
{
    "dataset": "customers"
}
```

but forgets:

```
question
```

FastAPI can automatically reject the invalid request.

This keeps invalid data from reaching deeper business logic.

---

43. Error Handling

A production-quality flow should handle errors at each layer.

```
React
 ↓
Network Error?

FastAPI
```

```
 ↓
Invalid Request?

Dataset
 ↓
Dataset Missing?

Agent
 ↓
Tool Failure?

Pandas
 ↓
Column Missing?

FAISS
 ↓
No Relevant Context?

LLM
 ↓
Generation Failure?

Validation
 ↓
Ground Truth Missing?
```

The backend should return structured errors rather than exposing internal stack traces.

For example:

```
{
    "error": "COLUMN_NOT_FOUND",
    "message": "The requested column does not exist in the selected dataset."
}
```

---

44. One Important Security Concern — Pandas Agent

This is a strong advanced interview point.

If an LLM-generated agent is allowed to dynamically generate and execute Python/Pandas code, that can become dangerous.

For example, malicious instructions could potentially try to perform operations outside the intended dataset scope.

Therefore, production systems should consider:

```
Sandboxing
Restricted execution
Tool allowlists
Input validation
Resource limits
Timeouts
Read-only data access
Container isolation
```

The principle is:

> Never give an LLM unrestricted code execution in a production environment.

---

45. Authentication Flow

Conceptually:

```
User
 ↓
Clerk
 ↓
Authenticated
 ↓
Frontend receives session/token
 ↓
Frontend calls FastAPI
 ↓
Backend validates identity
 ↓
Protected QueryLensAI endpoint
```

The frontend should not be considered the security boundary.

Backend endpoints should verify authentication independently.

---

46. Scaling QueryLensAI

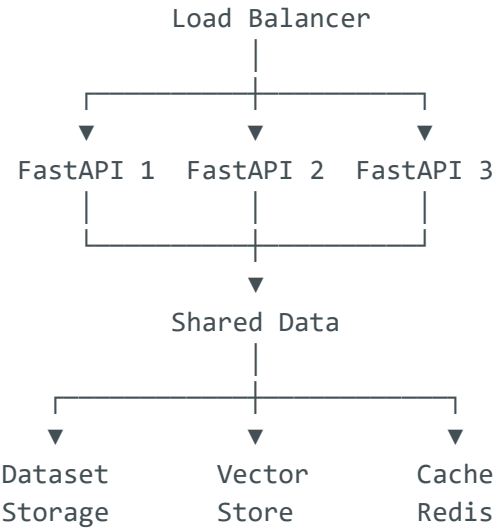
Suppose initially:

```
100 users
```

Later:

```
100,000 users
```

Potential architecture:



Possible improvements:

- Horizontal FastAPI scaling
- Precomputed embeddings
- Cached FAISS indexes
- Redis caching
- Background task queues
- Rate limiting
- Streaming responses
- Centralized logging
- Monitoring
- Containerization
- Cloud deployment

---

47. Why Cache FAISS Indexes?

Creating embeddings and rebuilding indexes repeatedly is expensive.

Instead:

```
Dataset uploaded
      ↓
Build embeddings
      ↓
Create FAISS index
      ↓
Cache/store index
```

Future queries:

```
Question
   ↓
Load existing index
   ↓
Similarity search
```

instead of:

```
Question
   ↓
Rebuild everything
   ↓
Search
```

This improves latency.

---

48. How Could We Improve Response Time?

Possible improvements:

```
Cache frequently used datasets

Precompute embeddings

Cache FAISS indexes
```

```
Use async FastAPI endpoints

Move long-running work to background workers

Stream LLM responses

Cache repeated questions

Use connection pooling

Avoid repeatedly loading datasets

Add indexes to persistent databases
```

---

49. What If Two Users Ask Questions Simultaneously?

FastAPI can handle multiple concurrent requests.

Each request should have its own isolated context:

```
User A
   ↓
Request A
   ↓
Dataset A

User B
   ↓
Request B
   ↓
Dataset B
```

The backend must avoid accidentally sharing user-specific state between requests.

For multi-user systems, dataset access should be scoped to the authenticated user or tenant.

---

50. Multi-Tenant Security

Suppose:

```
Company A uploads Dataset A

Company B uploads Dataset B
```

Company A must never retrieve:

```
Company B's data
```

So every resource should be associated with something like:

```
tenant_id
user_id
dataset_id
```

Then every backend operation verifies ownership/access.

Conceptually:

```
Authenticated User
       ↓
Tenant
       ↓
Dataset Permission
       ↓
Query Allowed
```

---

51. How Would You Monitor Wrong Answers?

Because QueryLensAI already produces:

```
valid_answer
reason
```

we can record metrics such as:

```
Total queries

Valid answers

Invalid answers

Validation failure rate

Failures by dataset

Failures by question type

FAISS retrieval quality

Agent routing failures

LLM latency

Pandas execution time
```

For example:

```
Accuracy Rate
=
Valid Answers / Total Answers
```

This helps identify regressions.

---

52. What If FAISS Retrieves the Wrong Context?

Possible reasons:

```
Poor embeddings

Bad chunking

Ambiguous query

Insufficient metadata

Wrong top-k

Weak similarity threshold
```

Possible improvements:

```
Better embedding model

Metadata filtering

Hybrid search

Reranking

Tune top-k

Similarity threshold

Better chunking

Query rewriting
```

---

53. What is Top-K?

Suppose FAISS finds 100 potentially related chunks.

We don't want to send all 100 to the LLM.

Instead:

```
top_k = 3
```

means:

```
Return the 3 most similar results.
```

Example:

```
Query:
"What does customer income mean?"

FAISS results:

1. Income represents annual salary       0.94
2. Income is stored in USD               0.87
3. Customer salary field documentation   0.83
```

```
4. Population definition                0.31
...
```

With:

```
top_k = 3
```

we use the first three.

---

54. What is Similarity Search?

Similarity search answers:

> Which stored vectors are closest to the query vector?

Common similarity/distance measures include:

```
Cosine similarity
Euclidean distance
Dot product
```

The exact method depends on how the index is configured.

The key concept is:

```
Question embedding
        ↓
Compare against stored embeddings
        ↓
Retrieve nearest vectors
```

---

55. What is Hallucination?

Hallucination is when an LLM generates information that sounds plausible but is not supported by the available facts.

Example:

Dataset says:

```
Average income = 15,000
```

LLM says:

```
Average income = 18,000
```

That is incorrect.

QueryLensAI reduces this risk through:

```
Real dataset computation
+
Retrieval
+
Ground-truth validation
```

---

56. Why Is Ground Truth Manually Curated?

For important benchmark questions, manually verified answers provide a reliable reference.

For example:

```
Question:
"What is the average income?"

Verified answer:
15000
```

Then generated outputs can be evaluated against known expected results.

This can help with:

```
Accuracy evaluation

Regression testing

Model comparison

Prompt improvement

Trust indicators
```

---

57. Limitation of Ground-Truth Validation

This is important.

Manual ground truth doesn't automatically scale to every possible natural-language question.

If users can ask unlimited questions, manually creating reference answers for every possible query becomes difficult.

Potential future approaches could include:

```
Programmatic validators

Deterministic execution results

Schema-aware validation

LLM-as-judge with safeguards

Automated benchmark generation

Confidence scoring
```

But an LLM judge should not automatically be treated as absolute truth.

---

58. What If the Agent Chooses the Wrong Tool?

Example:

User asks:

```
"What is the average income?"
```

But the agent chooses:

```
FAISS
```

instead of:

```
Pandas
```

This is a **routing error**.

Possible solutions:

```
Explicit tool descriptions

Better prompts

Fine-tuned routing

Intent classifier

Rule-based safeguards

Evaluation datasets

Fallback logic
```

For deterministic patterns like:

```
average
sum
count
maximum
minimum
```

the system can also use additional routing safeguards.

59. Agent vs Hardcoded Routing

Hardcoded

```
if "average" in query:
    pandas()
```

Advantages:

```
Simple
Fast
Predictable
Cheap
```

Disadvantages:

```
Rigid
Doesn't understand complex language
Hard to maintain as query types grow
```

Agent

```
Question
   ↓
LLM reasoning
   ↓
Tool selection
```

Advantages:

```
Flexible
Handles natural language
Can combine tools
```

Disadvantages:

```
More latency
More cost
Less deterministic
Possible incorrect routing
```

A robust system can combine both approaches.

---

60. Why Agentic AI?

Traditional application:

```
Input
 ↓
Predefined code path
 ↓
Output
```

Agentic workflow:

```
Input
 ↓
Understand objective
 ↓
Choose action/tool
 ↓
Observe result
 ↓
Potentially choose another action
 ↓
Generate answer
```

The system has some decision-making capability over which tools to invoke.

---

61. How Would You Explain QueryLensAI to a Non-Technical Person?

> QueryLensAI lets someone talk to their data in normal English. Instead of writing code to calculate something, they can simply ask a question. The system figures out whether it needs to search for information or calculate something from the dataset, performs that operation, explains the answer in normal language, and checks whether the answer is correct before showing it.

---

62. 30-Second Interview Explanation

> QueryLensAI is a conversational data-querying platform where users can query structured datasets using natural language instead of writing SQL or Pandas code. The React frontend communicates with a FastAPI backend, where LangChain orchestrates an agent that decides whether a query requires semantic retrieval through FAISS, deterministic computation through Pandas, or both. An LLM then converts the retrieved or computed result into a natural-language response. The interesting part is that the generated answer is validated against curated ground truth and returned with a Valid or Invalid status and reason.

---

63. 60-Second Interview Explanation

> QueryLensAI allows users to select a structured dataset and ask questions about it in plain English. The frontend is built using React 18 and Vite with Clerk authentication and Tailwind/DaisyUI, and it communicates with a FastAPI backend through REST APIs.
>
> On the backend, LangChain acts as the orchestration layer. An agent analyzes the user's query and determines whether it needs semantic retrieval using FAISS, actual structured-data computation using Pandas, or a combination of both. FAISS allows us to retrieve semantically relevant context using vector embeddings, while Pandas handles deterministic calculations such as filtering, averages, counts, and aggregations against the real dataset.
>
> The results are then converted into a natural-language response by the LLM. Finally, the answer is checked against a manually curated ground-truth reference, and the frontend receives the answer together with a Valid or Invalid status and a plain-English reason.

---

64. If Interviewer Asks: "Why LangChain?"

> We used LangChain as the orchestration layer between the LLM and external tools. It allows the system to decide whether a query requires semantic retrieval through FAISS, computation through Pandas, or multiple steps, instead of sending every question directly to the LLM.

---

65. If Interviewer Asks: "What is an Agent?"

> An agent is an LLM-driven decision-making component that can analyze the user's request and choose which available tool or action should be used to complete it.

---

66. If Interviewer Asks: "What is FAISS?"

> FAISS is a vector similarity-search library. We use it to retrieve semantically relevant information by comparing the embedding of the user's query with stored embeddings.

---

67. If Interviewer Asks: "What is an Embedding?"

> An embedding is a numerical vector representation of information where semantically similar content tends to have similar vector representations.

---

68. If Interviewer Asks: "What is RAG?"

> RAG stands for Retrieval-Augmented Generation. Instead of asking the LLM to answer purely from its pretrained knowledge, we first retrieve relevant external context and provide that context to the model when generating the answer.

---

69. If Interviewer Asks: "FAISS vs RAG?"

> FAISS is a technology that performs vector similarity retrieval. RAG is the broader architecture where retrieved information is provided to an LLM before generating an answer. FAISS can therefore implement the retrieval part of a RAG pipeline.

---

70. If Interviewer Asks: "Why Pandas?"

> Pandas gives us deterministic computation over the actual structured dataset. We use it for filtering, aggregation, averages, counts and similar operations instead of expecting the LLM to calculate those values reliably.

---

71. If Interviewer Asks: "Why Not Just Use the LLM?"

> Because LLMs are probabilistic and can hallucinate or make calculation errors. We use deterministic tools like Pandas for computation and FAISS for retrieval, while the LLM focuses mainly on understanding the query and generating a natural-language explanation.

---

72. If Interviewer Asks: "Why FastAPI?"

> FastAPI works well for Python-based AI backends because it provides type-safe request validation through Pydantic, async support, automatic API documentation and straightforward integration with Python AI and data libraries.

---

73. If Interviewer Asks: "Why React?"

> React provides a component-based architecture that works well for an interactive chat-style interface where UI elements such as messages, loading states, dataset selectors and validation badges update dynamically.

---

74. If Interviewer Asks: "What Was Technically Interesting?"

A strong answer:

> The interesting part wasn't simply calling an LLM. The challenge was deciding when the system should retrieve contextual information versus when it should perform deterministic computation against the actual dataset. That separation reduces reliance on the LLM for factual calculations, and the ground-truth validation layer gives us another mechanism for detecting incorrect generated answers.

---

75. If Interviewer Asks: "What Was a Challenge?"

One good direction, **only if it reflects what you actually worked on**, is:

> One challenging area was routing user questions correctly. Some questions are purely semantic retrieval, some require numerical computation, and others require both. The challenge is making sure the agent selects the correct tool rather than asking the LLM to answer everything directly.

Do not claim this was your personal implementation challenge unless it actually was.

---

76. If Interviewer Asks: "How Would You Scale It?"

> I would keep the FastAPI backend stateless so we can horizontally scale it behind a load balancer. I would cache dataset metadata and precomputed FAISS indexes, move expensive processing into background workers where appropriate, introduce Redis for caching, add rate limiting, and isolate user or tenant datasets. I would also monitor latency separately for retrieval, computation and LLM generation.

---

77. If Interviewer Asks: "How Would You Make It Production Ready?"

I would focus on:

```
Authentication
Authorization

Tenant isolation

Input validation

Rate limiting

Timeouts

Retries

Restricted agent tools

Sandboxed code execution

Caching

Background processing

Logging

Metrics

Tracing

Error handling

Model monitoring

Validation monitoring

Horizontal scaling

CI/CD
```

---

78. If Interviewer Asks: "What If the LLM Fails?"

> I would treat the LLM as an external dependency. The backend should use timeouts, controlled retries for transient failures, structured error handling and potentially fallback behavior. We should also log failures with request IDs so they can be traced without exposing sensitive dataset contents.

---

79. If Interviewer Asks: "What If FAISS Returns Nothing Useful?"

> I would use a similarity threshold so low-quality results aren't blindly passed to the model. Depending on the query, the system could retry with query rewriting, use another retrieval strategy, fall back to structured computation, or tell the user that sufficient context wasn't found rather than hallucinating an answer.

---

80. If Interviewer Asks: "How Do You Reduce Hallucination?"

> We reduce hallucination by grounding responses in retrieved context, using deterministic Pandas operations for calculations instead of relying on the LLM, and validating generated answers against known ground-truth references where available.

---

81. If Interviewer Asks: "How Would You Test This?"

I would test different layers independently:

```
Frontend Tests
     ↓
UI state and API integration

API Tests
     ↓
Request validation and responses

Pandas Tests
     ↓
Known dataset → known calculation

Retrieval Tests
     ↓
Question → expected relevant context

Agent Tests
     ↓
```

```
Question → expected tool selection

Validation Tests
     ↓
Generated answer → Valid/Invalid

End-to-End Tests
     ↓
Question → final UI response
```

For AI-specific evaluation, maintain benchmark questions containing:

```
Question
Expected Tool
Expected Result
Expected Validation
```

Then run them whenever prompts/models/tool descriptions change.

---

82. If Interviewer Asks: "How Would You Debug a Wrong Answer?"

Follow the pipeline.

```
Wrong Answer
     ↓
Was correct dataset selected?
     ↓
Was question interpreted correctly?
     ↓
Did agent select correct tool?
     ↓
Was FAISS retrieval relevant?
     ↓
Did Pandas calculate correctly?
     ↓
Was correct result passed to LLM?
     ↓
Did LLM change/distort the result?
     ↓
Did validation catch it?
```

This is much better than simply saying:

```
"The LLM gave a wrong answer."
```

The architecture allows us to isolate **where the failure occurred**.

---

83. Important Distinction — Retrieval vs Computation

This is probably the single most important technical distinction in the entire project.

Retrieval

```
Question:
"What does annual income mean?"

        ↓

FAISS

        ↓

Find relevant information
```

Computation

```
Question:
"What is average annual income?"

        ↓

Pandas

        ↓

Calculate actual value
```

Generation

```
Raw Result
        ↓

LLM

        ↓

Explain naturally
```

Validation

```
Generated Answer
        ↓

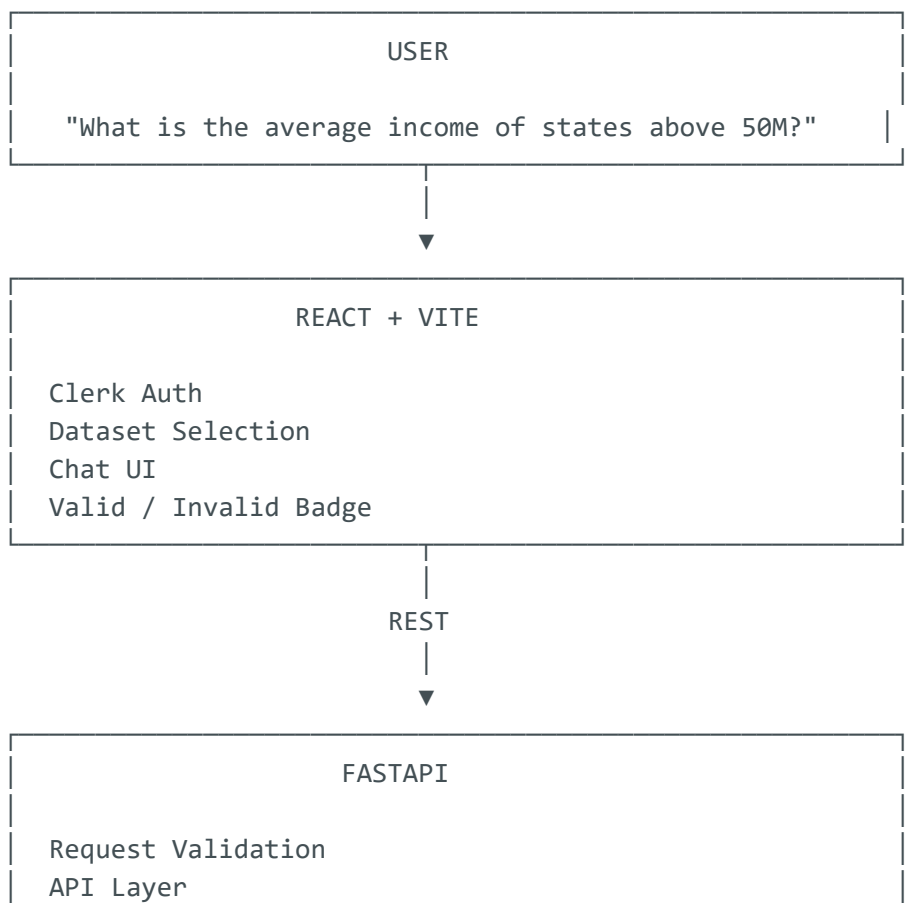
Ground Truth

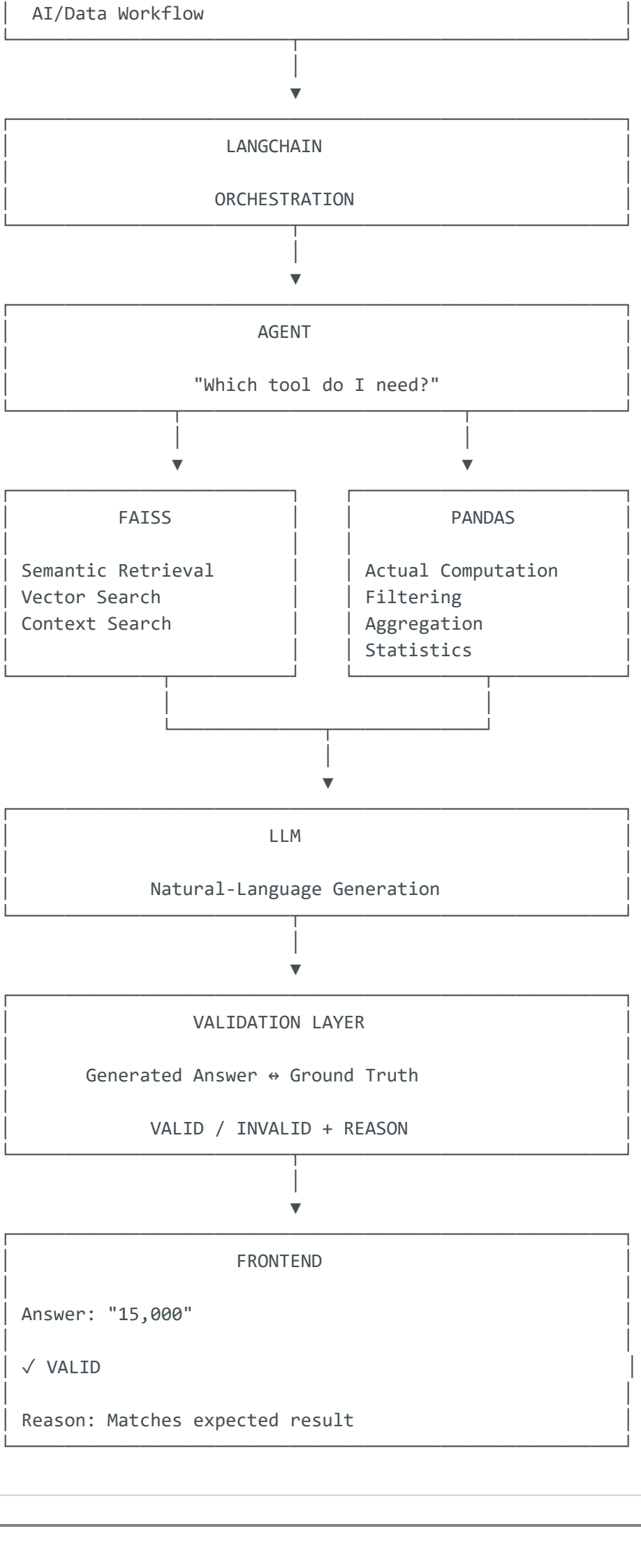
        ↓

Verify
```

---

84. QueryLensAI in One Diagram




AI/Data Workflow
LANGCHAIN
ORCHESTRATION
AGENT
"Which tool do I need?"
FAISS
Semantic Retrieval
Vector Search
Context Search
PANDAS
Actual Computation
Filtering
Aggregation
Statistics
LLM
Natural-Language Generation
VALIDATION LAYER
Generated Answer ⇔ Ground Truth
VALID / INVALID + REASON
FRONTEND
Answer: "15,000"
✓ VALID
Reason: Matches expected result

85. Final Interview Cheat Sheet

If you're immediately before an interview, memorize this:

Technology	One-Line Meaning
React	Builds the interactive frontend
Vite	Frontend development/build tool
Clerk	Authentication and user sessions
FastAPI	Python REST backend
Pydantic	Request/response validation
LangChain	Orchestrates LLMs, tools and workflows
Agent	Decides which tool/action to use
Embedding	Numerical representation of semantic meaning
FAISS	Searches similar vectors
RAG	Retrieve context before generating an answer
Pandas	Performs deterministic computation on datasets
LLM	Understands/generates natural language
Ground Truth	Known reference used to validate answers

---

86. Five Lines to Remember

If you forget everything else:

```
LangChain = ORCHESTRATE

Agent = DECIDE

FAISS = RETRIEVE

Pandas = CALCULATE

LLM = EXPLAIN
```

Then:

```
Ground Truth = VERIFY
```

Therefore the complete architecture becomes:

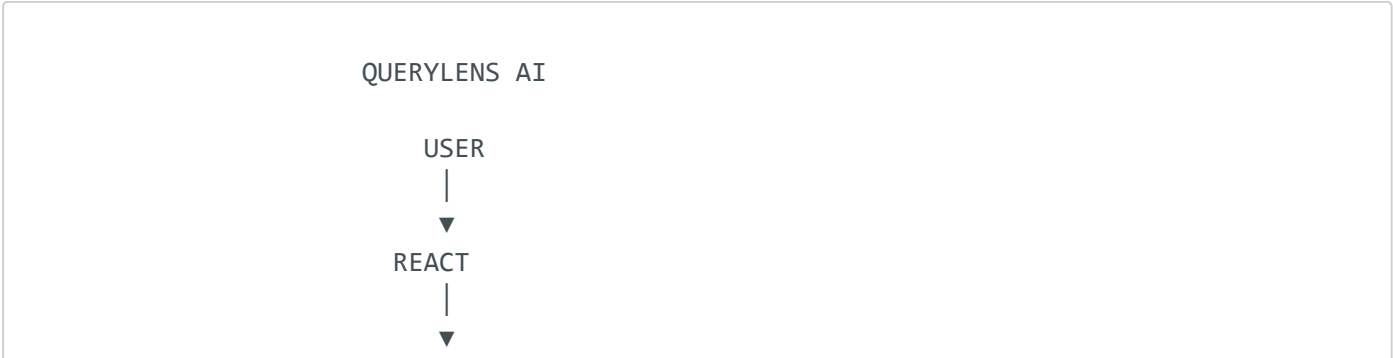
```
User Question
      ↓
ORCHESTRATE
      ↓
DECIDE
      ↓
RETRIEVE / CALCULATE
      ↓
EXPLAIN
      ↓
VERIFY
      ↓
Final Answer
```

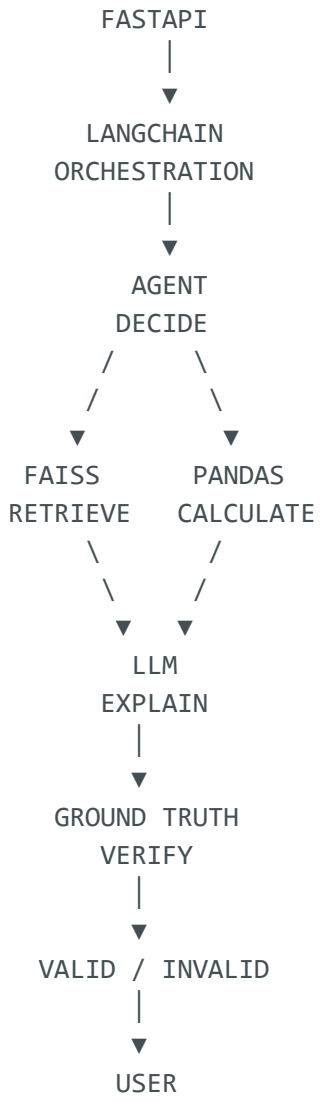
---

87. Best Interview Closing Explanation

> QueryLensAI isn't simply an LLM wrapper. The LLM is one component inside a larger data-processing pipeline. LangChain coordinates the workflow, the agent determines what capability is needed, FAISS retrieves semantically relevant context, Pandas performs deterministic computation against the actual dataset, and the LLM converts the result into natural language. We then add a validation layer that checks the generated response against curated ground truth and exposes the result as a Valid or Invalid status. That separation between retrieval, computation, generation and validation is the main architectural idea behind the project.

QueryLensAI — Mental Model





One sentence:

> **QueryLensAI converts natural-language questions into grounded data operations by intelligently choosing between semantic retrieval and deterministic computation, then generates and validates the final answer before presenting it to the user.**

AI Finance Platform

A full-stack AI-powered finance management platform built with **Next.js**, **Supabase**, **Prisma**, **Tailwind CSS**, **Shadcn UI**, **Clerk**, **Inngest**, **ArcJet**, and **Resend**.

The app is designed to help users **manage accounts, track income and expenses, scan receipts, handle recurring transactions, and visualize financial activity** in a clean, modern interface.

Overview

AI Finance Platform is a personal finance SaaS application that combines traditional financial tracking with AI-assisted workflows. It allows users to create accounts, add transactions, categorize spending, and analyze their money flow from a centralized dashboard.

The platform focuses on:

- **Ease of financial tracking**
- **Automation through AI**
- **Secure authenticated workflows**
- **Clean and scalable full-stack architecture**

Key Features

1. User Authentication

- Secure sign-in and sign-up flows powered by **Clerk**
- Protected routes for authenticated users
- User-specific dashboards and actions
- Seamless user session handling

2. Account Management

- Create and manage multiple financial accounts
- Support for account types like **Current** and **Savings**
- Set a default account for faster transaction entry
- Track balances per account

3. Transaction Management

- Add **income** and **expense** transactions
- Edit existing transactions
- Link transactions to specific accounts

- Categorize spending for better budgeting
- Support for recurring transactions
- Date-based transaction entry and updates

4. AI-Assisted Receipt Scanning

- Scan receipts to auto-fill transaction data
- Reduce manual entry for amounts, descriptions, dates, and categories
- Improve transaction capture speed and accuracy

5. Dashboard & Financial Overview

- Central dashboard for viewing financial data
- Account-level and transaction-level insights
- Visual structure ready for charts and analytics
- Designed for quick money management at a glance

6. Notifications & Email

- Transactional email support using **Resend**
- Useful for onboarding, alerts, and user communication
- Server-side email action for secure delivery

7. Background Workflows

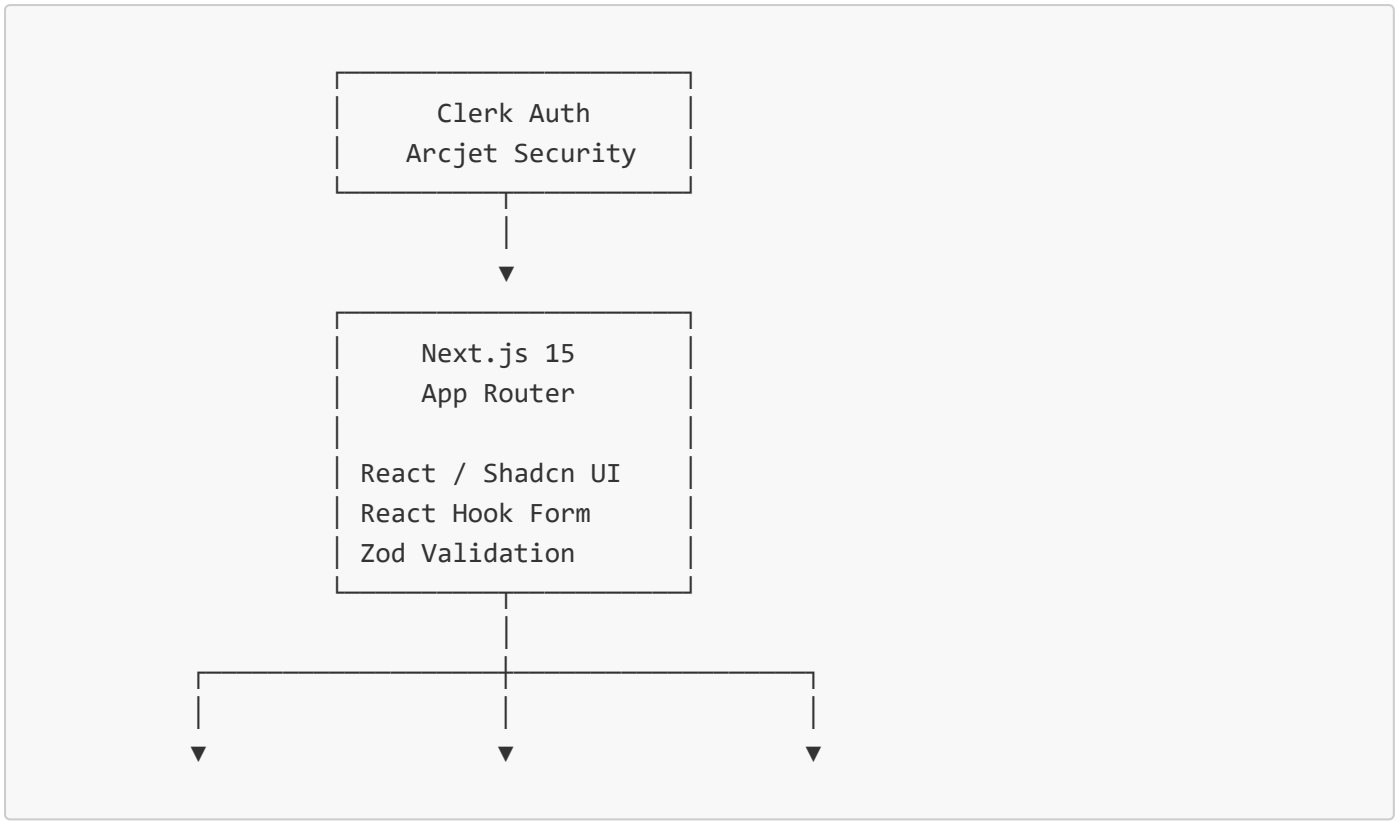
- Async task orchestration using **Inngest**
- Built-in retry strategy for reliability
- Suitable for workflows like notifications, processing, and automation

8. Security & Protection

- **ArcJet** for request protection and abuse prevention
- Rate limiting and safety measures for a finance-oriented app

---

Architecture



Tech Stack

Frontend

- **Next.js** — App Router-based full-stack React framework
- **React** — UI component model
- **Tailwind CSS** — Utility-first styling
- **Shadcn UI** — Reusable, accessible UI components

- **Lucide React** — Icon library

Backend / Data

- **Supabase** — Backend infrastructure and PostgreSQL database
- **Prisma** — Type-safe database ORM
- **Next.js Server Actions** — Secure server-side mutations
- **Inngest** — Event-driven background jobs

Authentication / Security

- **Clerk** — Authentication and user management
- **ArcJet** — Security, protection, and rate limiting

Forms / Validation

- **React Hook Form** — Form handling
- **Zod** — Schema validation
- **@hookform/resolvers** — Resolver integration

Notifications / Email

- **Resend** — Email delivery

Utilities

- **date-fns** — Date formatting and manipulation
- **sonner** — Toast notifications

How It Works

1. A user signs in using Clerk
2. The user creates or selects a financial account
3. The user adds income or expense transactions

4. Receipt scanning can auto-fill transaction fields
5. Data is saved through server actions and Prisma
6. The dashboard reflects updated financial information
7. Background tasks can run through Inngest
8. Emails can be sent using Resend

Design Principles

This project is built around a few key engineering ideas:

- **User-first UX** — minimize friction when tracking money
- **Secure-by-default** — protect user data and routes
- **Modular architecture** — separate UI, actions, schemas, and integrations
- **Scalability** — use background jobs and server actions for clean growth
- **Modern frontend patterns** — App Router, Server Components, and composable UI

Interview Highlights

If you're presenting this project in an interview, these are the strongest points to mention:

- Built a **full-stack SaaS-style finance application**
- Integrated **authentication, database, server actions, and background jobs**
- Implemented **AI-assisted receipt scanning** to reduce manual data entry
- Designed a **scalable architecture** with Next.js App Router and server/client component separation
- Used **Prisma + Supabase** for a type-safe and scalable data layer
- Added **transactional emails** and **security protections** for production readiness
- Focused on **UX, reliability, and maintainability**

Example interview summary

> "I built a full-stack AI finance platform that helps users manage accounts and transactions, scan receipts, and analyze spending. The app uses Next.js, Clerk,

> Prisma, Supabase, Inngest, ArcJet, and Resend, with a UI built in Tailwind and Shadcn. It combines secure authentication, server actions, background workflows, and AI-assisted data entry into a production-style personal finance product.”

Future Improvements

Potential next enhancements could include:

- Budget planning and spending goals
- Advanced charts and analytics
- Export to CSV/PDF
- Automated recurring bill detection
- Multi-currency support
- Investment tracking
- Push notifications
- More AI-driven financial insights
